

编译原理 第十二次作业

姓名：朱首赫

学号：2023K8009906029

1 练习 8.1.1

假设 n 在一个内存位置中， s 、 i 分配在寄存器中，为下面的语句序列生成代码，并计算生成的目标代码的代价（其中访存代价为 3，分支代价为 2，其他指令代价为 1）

```
s = 0
i = 0
L1:
    if i >= n goto L2
    s = s + i
    i = i + 2
    goto L1
L2:
```

基本假设：

- 变量 s 已分配在寄存器 R_s 中。
- 变量 i 已分配在寄存器 R_i 中。
- 变量 n 存放在内存中，设其内存地址符号也为 n 。
- 使用一个通用寄存器 R_n 用于在循环中临时存放从内存中读取的 n 的值。
- 使用一个临时寄存器 R_t 用于存放比较时的差值。

1. 目标代码生成与单条指令代价分析

根据题目给定的代价模型：访存代价 = 3，分支代价 = 2，其他指令代价 = 1。将源程序翻译为标准的三地址/汇编指令集：

源程序	目标代码	指令类型	代价
$s = 0$	LD $R_s, 0$	寄存器赋值（其他）	1
$i = 0$	LD $R_i, 0$	寄存器赋值（其他）	1
L1:	L1:	标号（无指令）	0
if $i \geq n$ goto L2	LD R_n, n	内存加载（访存）	3
	SUB R_t, R_i, R_n	运算指令（其他）	1
	BGEZ $R_t, L2$	条件跳转（分支）	2
$s = s + i$	ADD R_s, R_s, R_i	算术加法（其他）	1
$i = i + 2$	ADD $R_i, R_i, 2$	算术加法（其他）	1
goto L1	BR L1	无条件跳转（分支）	2
L2:	L2:	标号（无指令）	0

2. 目标代码代价计算

静态总代价（所有生成的机器指令代价之和）：

$$\text{Cost}_{\text{static}} = 1 + 1 + 3 + 1 + 2 + 1 + 1 + 2 = 12$$

补充说明（动态代价）：如果考查的是程序运行时的动态执行代价，则需要考虑循环次数。假设 $n > 0$ ，循环体每次 i 步进 2，共循环 $\lceil n/2 \rceil$ 次：

- 初始化指令代价： $1 + 1 = 2$
- 循环体内部执行代价： $(3 + 1 + 2 + 1 + 1 + 2) \times \lceil n/2 \rceil = 10 \times \lceil n/2 \rceil$
- 最后一次跳出循环的判断代价：加载与减法 $3 + 1 = 4$ ，跳转生效 2，共计 6

总动态代价公式为：

$$\text{Cost}_{\text{dynamic}} = 10 \times \lceil n/2 \rceil + 8$$

2 练习 8.1.2

假设使用栈式分配，且假设 a 和 b 都是元素大小为 4 字节的数组，为下面的三地址语句生成代码

```
x = a[i]
y = b[j]
a[i] = y
b[j] = x
```

基本假设：

- 栈帧结构：假设使用栈指针寄存器 SP （Stack Pointer）。所有的局部变量和数组都分配在当前栈帧内。
- 变量偏移：设标量 x, y, i, j 相对于 SP 的内存地址偏移量分别为 O_x, O_y, O_i, O_j 。
- 数组偏移：设数组 a 和 b 的起始基址相对于 SP 的偏移量分别为 O_a 和 O_b 。
- 寄存器：假设 R_1, R_2, R_3 为可用的通用寄存器。

目标代码生成：

三地址语句	生成的目标代码	代码逻辑说明
$x = a[i]$	$LD\ R_1, O_i(SP)$ $MUL\ R_1, R_1, 4$ $ADD\ R_2, SP, R_1$ $LD\ R_3, O_a(R_2)$ $ST\ O_x(SP), R_3$	将下标 i 加载到 R_1 计算数组元素字节偏移量 ($i \times 4$) R_2 存入当前基础偏移: $SP + i \times 4$ 加载地址 ($SP + i \times 4$) + O_a (即 $a[i]$) 到 R_3 将 R_3 存入变量 x 的内存位置
$y = b[j]$	$LD\ R_1, O_j(SP)$ $MUL\ R_1, R_1, 4$ $ADD\ R_2, SP, R_1$ $LD\ R_3, O_b(R_2)$ $ST\ O_y(SP), R_3$	将下标 j 加载到 R_1 计算数组元素字节偏移量 ($j \times 4$) R_2 存入当前基础偏移: $SP + j \times 4$ 加载地址 ($SP + j \times 4$) + O_b (即 $b[j]$) 到 R_3 将 R_3 存入变量 y 的内存位置
$a[i] = y$	$LD\ R_1, O_i(SP)$ $MUL\ R_1, R_1, 4$ $ADD\ R_2, SP, R_1$ $LD\ R_3, O_y(SP)$ $ST\ O_a(R_2), R_3$	将下标 i 加载到 R_1 计算数组元素字节偏移量 ($i \times 4$) R_2 存入当前基础偏移: $SP + i \times 4$ 将变量 y 的值加载到 R_3 将 R_3 存回地址 ($SP + i \times 4$) + O_a 即 $a[i]$
$b[j] = x$	$LD\ R_1, O_j(SP)$ $MUL\ R_1, R_1, 4$ $ADD\ R_2, SP, R_1$ $LD\ R_3, O_x(SP)$ $ST\ O_b(R_2), R_3$	将下标 j 加载到 R_1 计算数组元素字节偏移量 ($j \times 4$) R_2 存入当前基础偏移: $SP + j \times 4$ 将变量 x 的值加载到 R_3 将 R_3 存回地址 ($SP + j \times 4$) + O_b 即 $b[j]$

3 练习 8.1.3

考虑下面的矩阵乘法程序：

1. 假设每个矩阵元素占 4 字节，且矩阵按行存放，把程序翻译成本节中的三地址语句并标出基本块
2. 为 1. 中得到的代码构造流图
3. 找到 2. 中流图中的循环

```

1  for (i=0; i<n; i++)
2      for (j=0; j<n; j++)
3          c[i][j] = 0.0;
4
5  for (i=0; i<n; i++)
6      for (k=0; k<n; k++)
7          for (j=0; j<n; j++)
8              c[i][j] = c[i][j] + a[i][k]*b[k][j];

```

1. 翻译为三地址语句并标出基本块

对于按行存放的 $n \times n$ 矩阵，元素 $A[x][y]$ 相对于基址的字节偏移量为 $(x * n + y) * 4$ 。按照程序执行顺序列出三地址语句，并用 $B_1 \sim B_{15}$ 划分基本块如下。

第一部分：矩阵清零循环 (Initialization)

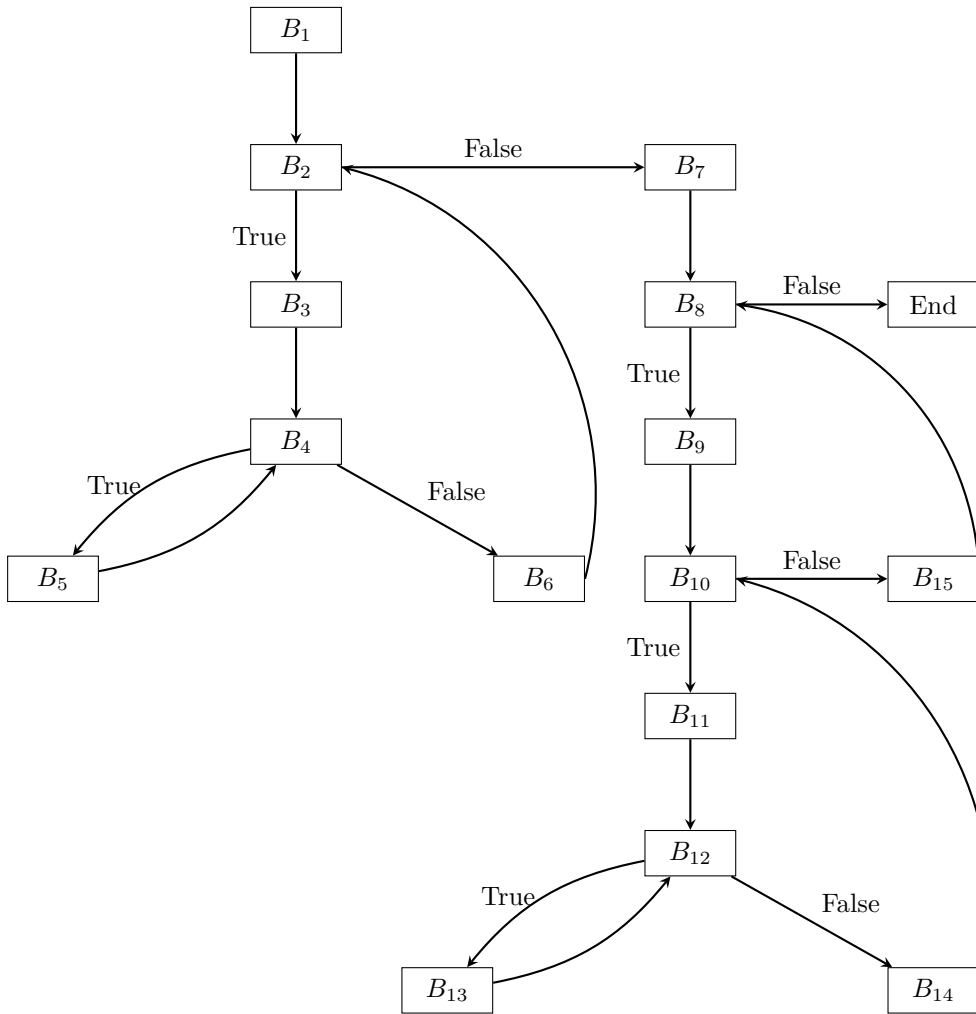
基本块	三地址语句
B_1	(1) $i = 0$
B_2	(2) if $i \geq n$ goto (13) // 跳出第一层外循环，进入 B_7
B_3	(3) $j = 0$
B_4	(4) if $j \geq n$ goto (11) // 跳出第一层内循环，进入 B_6
B_5	(5) $t1 = i * n$ (6) $t2 = t1 + j$ (7) $t3 = t2 * 4$ (8) $c[t3] = 0.0$ (9) $j = j + 1$ (10) goto (4) // 返回 B_4
B_6	(11) $i = i + 1$ (12) goto (2) // 返回 B_2

第二部分：矩阵乘法循环 (Multiplication)

基本块	三地址语句
B_7	(13) $i = 0$
B_8	(14) if $i \geq n$ goto END // 程序结束
B_9	(15) $k = 0$
B_{10}	(16) if $k \geq n$ goto (38) // 跳出中层循环, 进入 B_{15}
B_{11}	(17) $j = 0$
B_{12}	(18) if $j \geq n$ goto (36) // 跳出最内层循环, 进入 B_{14}
B_{13}	(19) $t4 = i * n$ // 以下计算 $c[i][j]$ 的偏移 $t6$ (20) $t5 = t4 + j$ (21) $t6 = t5 * 4$ (22) $t7 = i * n$ // 以下计算 $a[i][k]$ 的偏移 $t9$ (23) $t8 = t7 + k$ (24) $t9 = t8 * 4$ (25) $t10 = k * n$ // 以下计算 $b[k][j]$ 的偏移 $t12$ (26) $t11 = t10 + j$ (27) $t12 = t11 * 4$ (28) $t13 = c[t6]$ (29) $t14 = a[t9]$ (30) $t15 = b[t12]$ (31) $t16 = t14 * t15$ (32) $t17 = t13 + t16$ (33) $c[t6] = t17$ (34) $j = j + 1$ (35) goto (18) // 返回 B_{12}
B_{14}	(36) $k = k + 1$ (37) goto (16) // 返回 B_{10}
B_{15}	(38) $i = i + 1$ (39) goto (14) // 返回 B_8

2. 构造流图

根据上面的基本块划分，构造程序的控制流图如下。图中的节点对应上述基本块，有向边表示执行流。



3. 找到流图中的循环

根据流图中的**回边**（指向其自身或支配节点的边），可以定位到五个自然循环，正好对应原 C 代码中的五个 for 循环结构：

1. 回边 $B_5 \rightarrow B_4$ ：构成第一部分（初始化）的**内层循环**。

- 循环节点： $\{B_4, B_5\}$

2. 回边 $B_6 \rightarrow B_2$ ：构成第一部分（初始化）的**外层循环**。

- 循环节点： $\{B_2, B_3, B_4, B_5, B_6\}$

3. 回边 $B_{13} \rightarrow B_{12}$ ：构成第二部分（乘法）的**最内层循环**（变量 j 的循环）。

- 循环节点： $\{B_{12}, B_{13}\}$

4. 回边 $B_{14} \rightarrow B_{10}$ ：构成第二部分（乘法）的**中间层循环**（变量 k 的循环）。

- 循环节点： $\{B_{10}, B_{11}, B_{12}, B_{13}, B_{14}\}$

5. 回边 $B_{15} \rightarrow B_8$ ：构成第二部分（乘法）的**最外层循环**（变量 i 的循环）。

- 循环节点： $\{B_8, B_9, B_{10}, B_{11}, B_{12}, B_{13}, B_{14}, B_{15}\}$

4 练习 8.1.4

考虑右面的基本块

1. 构造 DAG

2. 假设只有 a 在基本块出口活跃，尝试优化右面的代码，并简述用到的技术

$d = b + c$

$e = b + c$

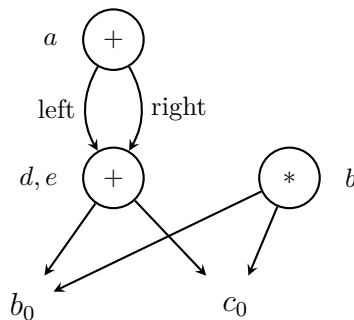
$b = b * c$

$a = e + d$

1. 构造 DAG (有向无环图)

构建 DAG 的过程如下：

- $d = b + c$ ：创建叶节点 b_0 和 c_0 ，然后创建一个操作符为 $+$ 的内部节点，其子节点为 b_0 和 c_0 ，并将该节点标记为 d 。
- $e = b + c$ ：查找发现操作符为 $+$ 且子节点为 b_0, c_0 的节点已存在。不需要创建新节点，直接将附加标记 e 加到已存在的 d 节点上。
- $b = b * c$ ：创建操作符为 $*$ 的内部节点，子节点为 b_0 和 c_0 ，并将该节点附加标记为 b 。
- $a = e + d$ ：创建操作符为 $+$ 的内部节点，它的两个操作数都是上面的 (d, e) 节点。将该节点附加标记为 a 。



2. 优化代码及简述用到的技术

根据题意，假设只有 a 在基本块出口处活跃（即只有变量 a 的值会在该基本块之后被使用）。基于我们构造的 DAG 以及活跃变量信息，可以进行以下优化：

优化后的代码：

$d = b + c$

$a = d + d$

用到的优化技术简述：

1. **公共子表达式消除**：由 DAG 的构造过程可知， $d = b + c$ 和 $e = b + c$ 计算的是同一个值，映射到了 DAG 中的同一个节点。因此， e 是一个多余的变量，后续使用 e 的地方都可以直接用 d 替换， $a = e + d$ 被替换为 $a = d + d$ 。
2. **死代码消除**：由于已知“只有 a 在基本块出口活跃”，这意味着变量 b （由 $b = b * c$ 产生）、 c 、 d 和 e 在离开这个基本块后都不会再被读取。

- 赋值语句 $b = b * c$ 计算出的新值在这之后没有任何语句使用它，因此这是一条**死语句**，可以直接安全地删除。
- 赋值语句 $e = b + c$ 在第一步消除公共子表达式后， e 也失去了被使用的机会，同样可以被消除。

综上所述，原先 4 条指令被高度精简为了 2 条指令。